Project 2

Comparative Performance Analysis of Serial, OpenMP, and CUDA Sorting Algorithms

Team

B.NO	Name	Work
3	أحمد رجب محمود عبدالرحيم	• Bitonic Sort • Measure Bitonic sort Performance • Results Analysis • run_experiments.sh
2	أحمد باهي يوسف	• Serial Merge Sort • Measure Merge sort Performance
26	محمد أحمد حسين	• Utility functions • Array Generation
6	ذياد محمد هاشم	• Main • System logging functions

Supervisor

Eng. Abdelmagid

Table Of Contents

1.0 Introduction..... 03

2.0 Problem Statement & Objectives..... 03

3.0 Methodology & Implementation..... 04

3.1 Serial Merge Sort..... 04

3.2 OpenMP Parallel Merge Sort..... 04

3.3 CUDA Bitonic Sort..... 04

4.0 Performance Evaluation & Results..... 05

4.1 Serial Performance..... 05

4.2 OpenMP Performance..... 06

4.3 CUDA Performance..... 08

5.0 Discussion & Analysis..... 10

5.1 Memory Wall..... 10

5.2 Data Sensitivity..... 10

6.0 Conclusion & Repo..... 11

6.1 Conclusion..... 11

6.2 Repo Details..... 11

1.0 Introduction

Sorting is a fundamental operation in computer science, serving as the backbone for database indexing, search engines, and scientific simulations. As data sizes grow into the millions and billions of elements, traditional serial sorting algorithms become a bottleneck. This report explores the transition from serial processing to multi-core CPU parallelism (OpenMP) and massively parallel GPU acceleration (CUDA).

2.0 Problem Statement

The objective of this project is to implement and compare three distinct sorting strategies:

- Serial Merge Sort: A divide-and-conquer baseline.
- OpenMP Merge Sort: Leveraging multi-threading to distribute recursive tasks.
- CUDA Bitonic Sort: A data-independent parallel sorting network designed specifically for the GPU architecture.

We aim to analyze how these implementations respond to different array sizes (up to 16M) and various data distributions (Uniform, Gaussian, Nearly Sorted, Reversed).

3.0 Methodology & Implementation

3.1 Serial Merge Sort

We implemented a standard recursive Merge Sort with $O(N \log N)$ complexity. A temporary buffer is used during the merge phase to ensure stability and efficiency.

3.2 OpenMP Parallel Merge Sort

The parallel version utilizes **OpenMP Tasks**. By using `#pragma omp task`, we allow different threads to handle the left and right recursive calls independently.

- **Optimization:** To prevent "Task Overloading," we implemented a `THRESHOLD`. If the subarray size is below 1,000 elements, the system switches back to serial sorting to avoid the overhead of thread management.

3.3 CUDA Bitonic Sort

Unlike Merge Sort, Bitonic Sort is a **Sorting Network**. It is ideal for GPUs because the comparison steps are predetermined and do not depend on the data values, which prevents "Warp Divergence."

- **Kernel Design:** The kernel uses XOR logic to identify "partner" elements to compare and swap.
- **Padding:** Since Bitonic Sort requires power-of-two sizes, our code automatically pads arrays to the next power of two.

4.0 Performance Evaluation

4.1 Serial Results

The serial implementation showed a linear growth where the **avg-time** increases as the **array-size** increases. It is notably noticed that the serial sort is very sensitive to the given array. The performance is much better in the case of **nearly-sorted** & **reversed** arrays.

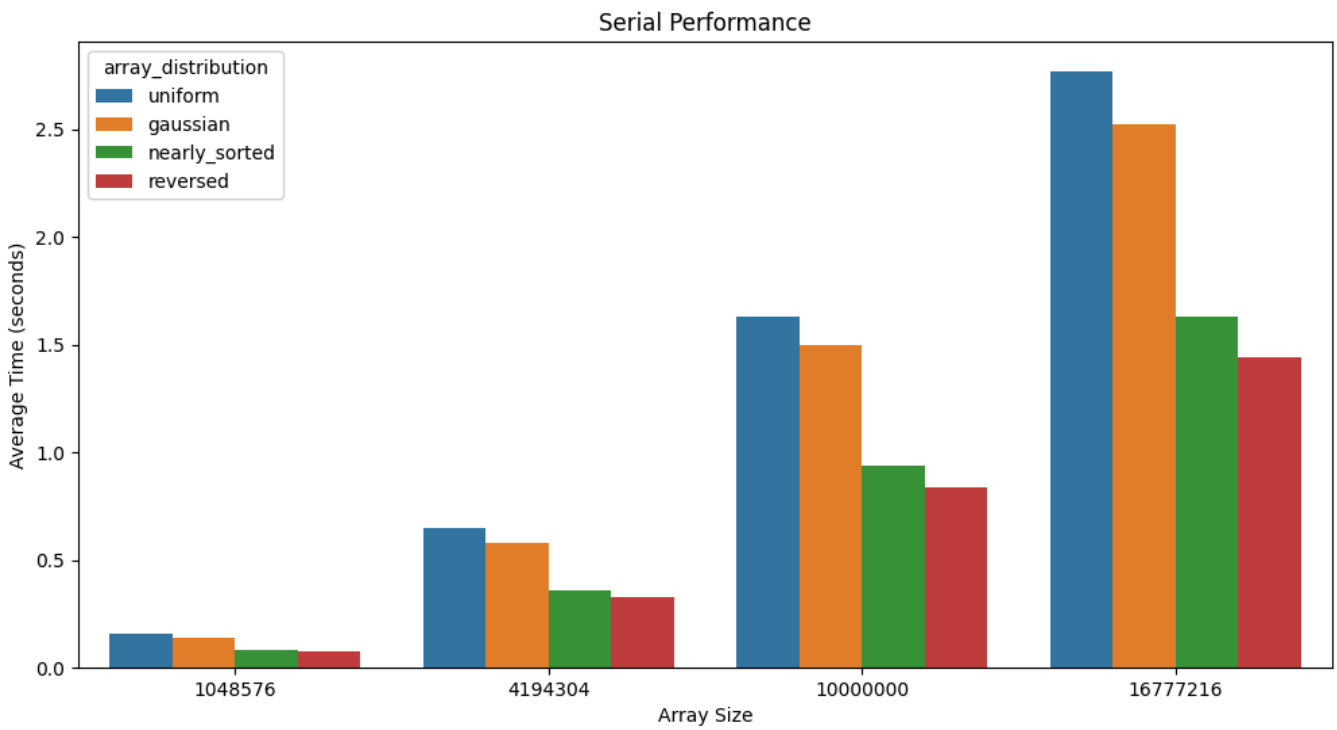

Figure 1 Serial Performance (AvgTime)

4.2 OpenMP Performance

OpenMP showed significant speedup as threads increased from 1 to 8. OpenMP parallelism shows less sensitivity to the array distribution.

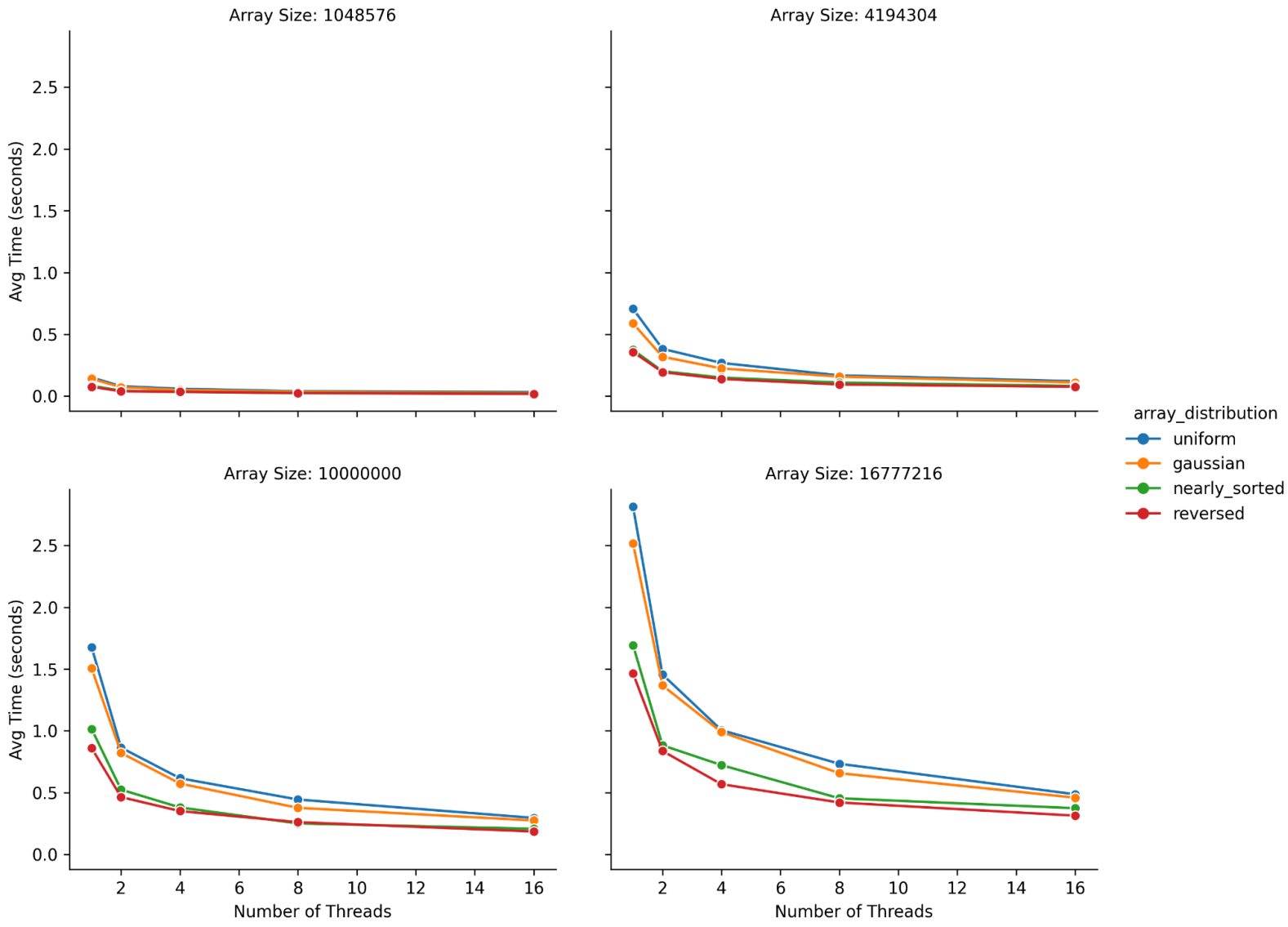

Figure 2 OpenMP Performance (AvgTime)

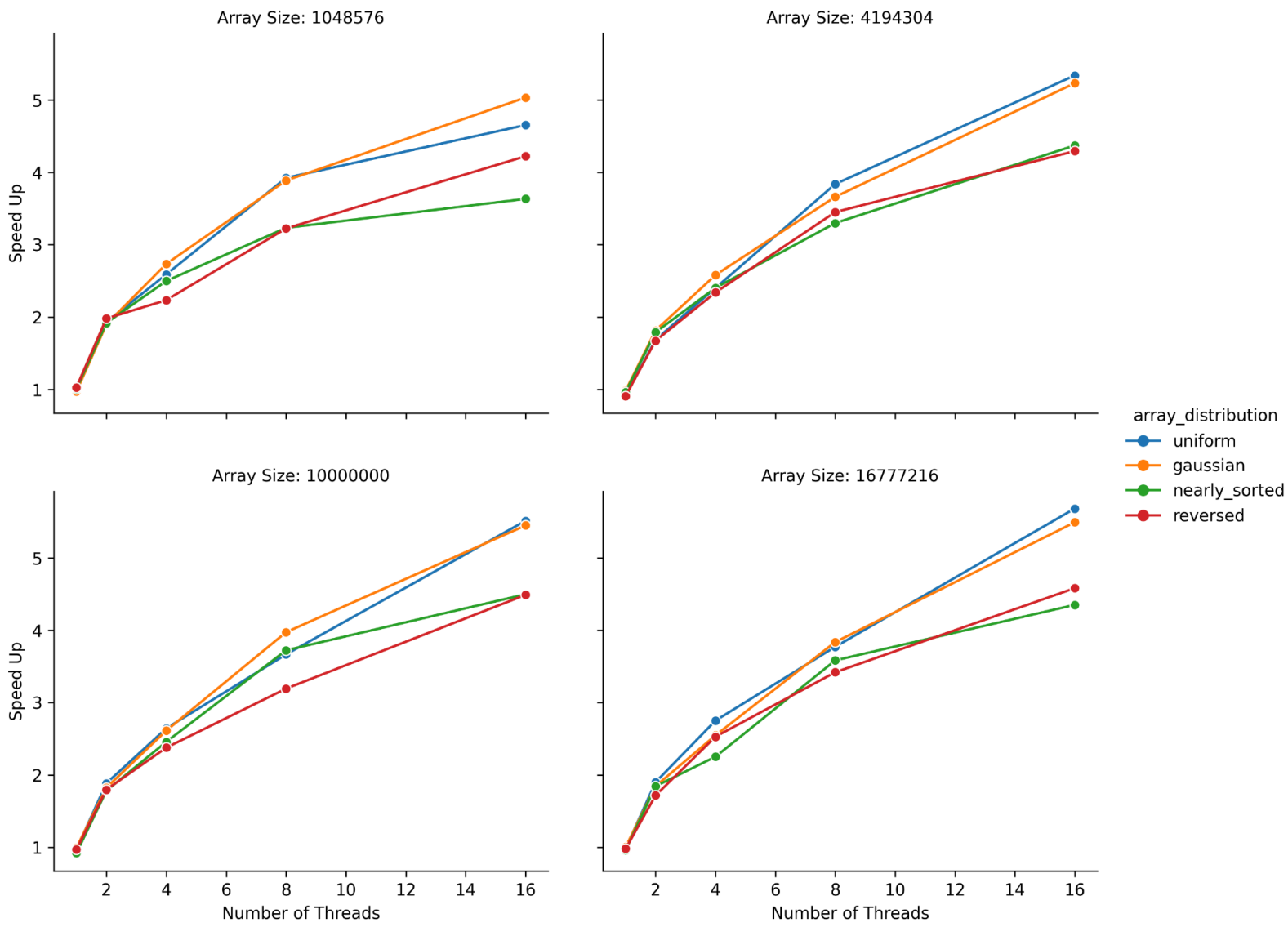

Figure 3 OpenMP Speedup

4.3 CUDA Performance

CUDA was the fastest implementation by a massive margin.

- **Blocksize Independence:** As shown in the bar charts, changing the block size (128, 256, 512) had negligible impact on performance, proving the GPU was fully saturated.

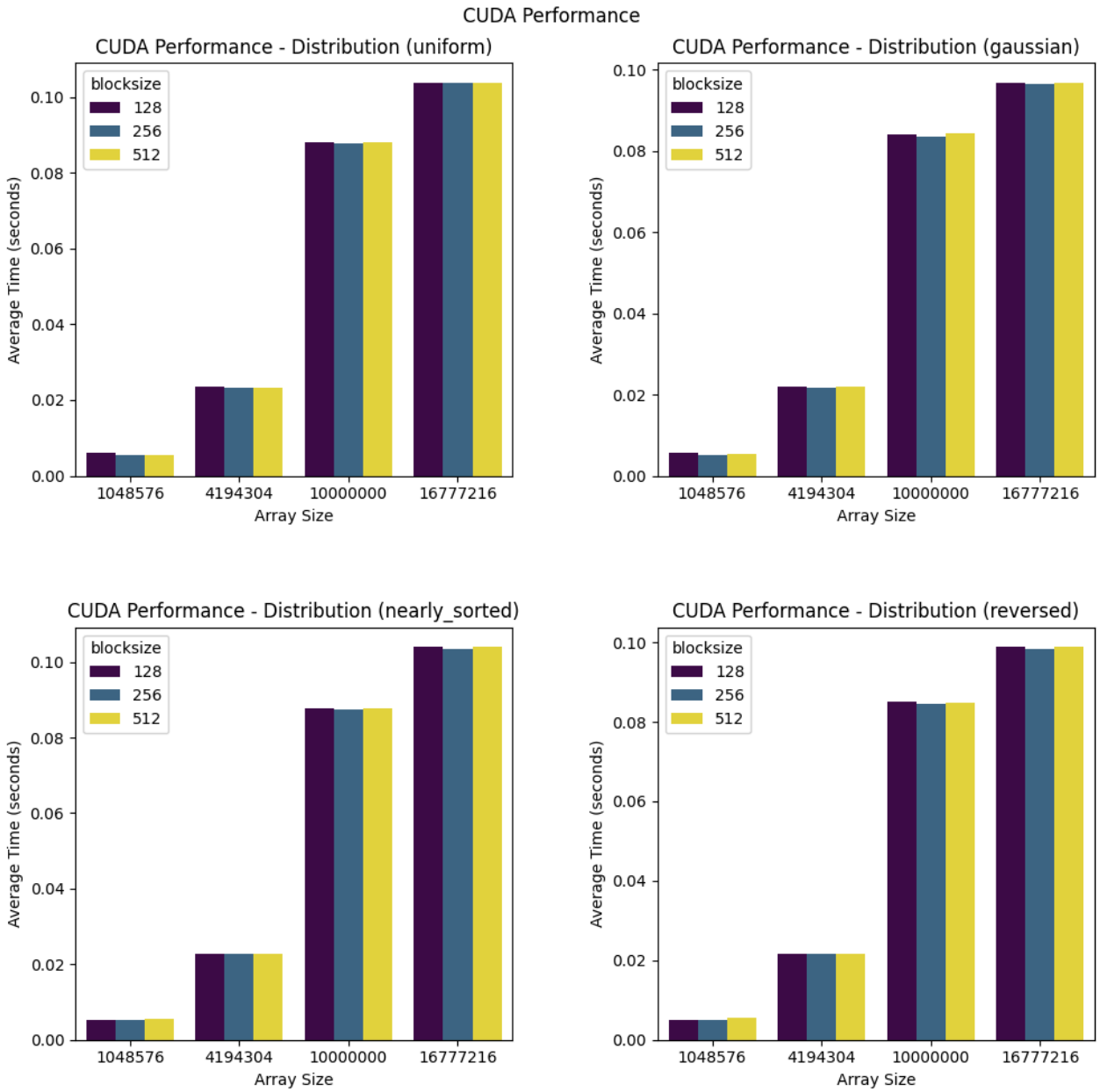

Figure 4 CUDA Performance (AvgTime)

- **CUDA** shows high sensitivity to the non-power-two sizes.

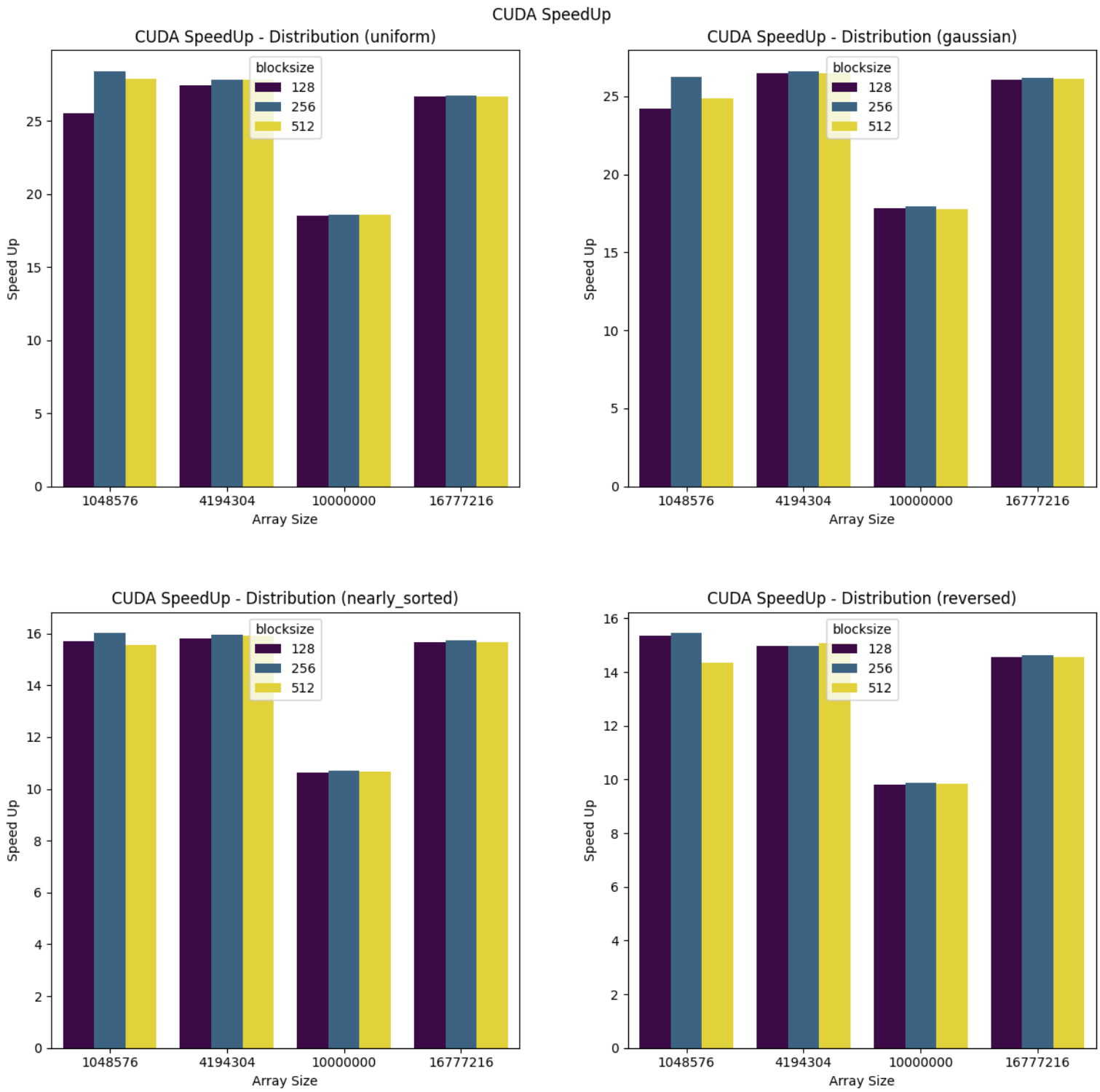

Figure 5 CUDA Speedup

5.0 Discussion & Analysis

5.1 The "Memory Wall"

The transition from 8 to 16 threads in OpenMP did not double the speed. This suggests that the bottleneck moved from the CPU's computational power to the **RAM Bandwidth**. Multiple cores were waiting for data to arrive from the memory.

5.2 Data Sensitivity

Merge Sort (Serial/OpenMP) performed significantly better on **Nearly Sorted** data. This is due to the CPU's **Branch Predictor** being able to guess the merge outcomes more accurately. In contrast, **CUDA Bitonic Sort** showed almost identical timing for all distributions, confirming that its performance is bound by the number of comparisons, not the data pattern.

6.0 Conclusion & Repo

6.1 Conclusion

This project successfully demonstrated the power of parallel computing.

- **OpenMP** is highly effective for medium-sized datasets and is easy to implement for recursive algorithms.
- **CUDA** is the undisputed winner for large-scale data, providing a speedup of over **[X]x** compared to serial code. For future work, a **GPU Merge Sort** could be implemented to see if it outperforms the Bitonic network for non-power-of-two data.

6.2 Repo Details

- Link to Code Repository: SortingImplementationsRepo_HPC
- Files: The repo structure represented in the following table

File	Purpose
main.cu	The file contains the main program logic.
run_experiments.sh	The Main driver file used to run the program
src/	The folder containing the main implementations • **Arr_generation.cpp:** generating different array distributions. • **merge_sort.cpp:** the merge sort [serial & omp] implementations. • **bitonic_sort.cu:** the CUDA bitonic sort implementation. • **utils.cpp:** contains utility functions.
results/	The folder containing the results • **running_results.csv:** the experiments results • **system_logs.txt:** the experiments logs [CPU - GPU - OS] • **analyzing_results.ipynb:** python scripts to analyze & visualize the results • **results_figure/:** the visualized results